

Towards AR for Large-Scale Robotics

Johannes Braumann*
Creative Robotics, UfG Linz
Association for Robots in Architecture

Emanuel Gollob
Creative Robotics, UfG Linz

Amir Bastan
Creative Robotics, UfG Linz

ABSTRACT

Today, robotic arms are widely used in industry and increasingly being adopted by small enterprises and startups where they perform tasks beyond the concept of mass fabrication that these machines were developed for. Facilitating the use of industrial robots through new, domain-specific interfaces is essential for fostering these applications. Augmented reality (AR) has got the potential to more closely connect the digital aspects of robot programming with the physical aspects of materiality and the synchronization of these workspaces. We present approaches that go beyond existing AR solutions, primarily using research-oriented or collaborative robots, towards integrating AR with large-scale industrial robots and their comparably dated software architecture. By integrating these strategies into a visual programming environment, they can be made available to people with domain-specific expertise but little programming knowledge, thus fostering innovation and digitization in these fields.

Keywords: augmented reality, robotic fabrication, human-robot interaction.

Index Terms: Computer systems organization~Embedded and cyber-physical systems~Robotics~External interfaces for robotics • Human-centered computing~Human computer interaction (HCI)~Interaction paradigms~Mixed / augmented reality

1 INTRODUCTION

We consider robotic arms to be interfaces that allow us to directly affect our physical world through digital data. In previous research we have developed strategies to map human motion capture data onto a robotic system consisting of five robotic arms (Figure 1) and created interfaces for robotic puppetry (Figure 2).

Doing so required in-depth knowledge of the robotic arms, their communication interfaces, and robot programming in general. Therefore, our research's underlying goal is to make robots increasingly accessible to new user groups, so that non-expert users may at some point define these experiences.

A significant challenge for new robot programmers is the synchronization between the digital and physical environment [6]. This research provides an overview of robot programming and outlines the complications of communicating with industrial machinery. As a proof of concept, this paper demonstrates the need for matching digital simulation and physical fabrication in real time, through a multiplatform approach; aiming for crafting 3D printed objects in an ephemeral workspace resulting site-specific objects. Visual programming is explored as a potential pathway towards making augmented reality and robotics more accessible to new groups of users.

* johannes@robotsinarchitecture.org

LEAVE 0.5 INCH SPACE AT BOTTOM OF LEFT
COLUMN ON FIRST PAGE FOR COPYRIGHT BLOCK

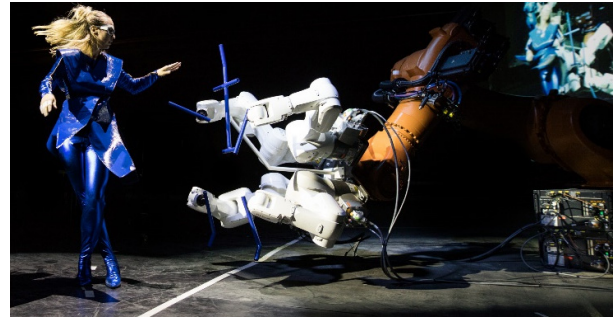


Figure 1: "underbody" project using motion capture data mapped to five synchronized robotic arms.



Figure 2: "pinocchio" puppetry project using two robotic arms controlled in real-time.

2 STATE OF THE ART

Robotic arms are today not only used for industrial mass fabrication but have become a multifunctional, reliable motion platform for a large variety of processes, from on-site production processes [7] to object manipulation [19] and multimedia experiences [4]. Their kinematic layout, usually providing six degrees of freedom, makes them flexible machines that can be equipped with a range of tools that theoretically can be moved to any position and orientation within the machine's workspace. Therefore, robotic arms form a foundation of Industry 4.0's cyber-physical systems, with concepts such as the "Digital Twin" proposing virtual commissioning of complex, robotic setup that can be explored and tested in virtual reality [11].

However, a highly significant challenge in robot programming is the synchronization between the physical and virtual environments – without it, processes programmed in the digital, zero-tolerance 3D space of CAX (computer-aided technologies) software cannot be accurately and reliably performed by the machine.

We see at least three levels on which such synchronization may take place:

- **Online Programming:** The first approach is to primarily stay within a physical environment: In industry, the most common way of robot programming is still teaching by demonstration [16], where the robot is physically moved to a position that is then saved in a control data file. Doing so allows the user to benefit from the robot's very high repeatability and to compensate for its worse

absolute accuracy. However, teaching a high number of positions is highly labor-intensive, making it less ideal for typical CNC (computer numeric control) fabrication processes, especially if they are intended for producing prototypes rather than mass fabrication.

- **Base Calibration:** In a digital environment, the generation of positions can be automated, for example to follow a given curve or to remove material with the goal of carving out a given shape through typical CAM (Computer-aided manufacturing) workflows. Within such an environment, the synchronization is often performed by measuring two coordinate systems: The base coordinate system, placing the origin-coordinate system of the CAD (Computer-aided design) environment within the robot's own world coordinate system, and the tool coordinate system, defining the TCP (tool center point) within the robot's flange coordinate system [12]. This calibration process first has the user approach an arbitrary point in space with the TCP from four different directions, which allows the robot to compute the offset of the TCP from the robot's flange. With that point defined, the robot's base coordinate system may be defined via three points in space: An origin point, a point in X-direction, and a point in Y-direction.

- **Handy-Eye Calibration:** As the robot's absolute axis position and kinematic layout are always known, the coordinate system of the robot and its digitized environment can be matched via markers that are placed either in the environment or on the robot by hand-eye calibration.

In practice, each of these approaches has its advantages and disadvantages. For example, code generated through online programming is highly optimized for the setup within which it was generated, often requiring the user to re-teach all positions if a significant change is made to the robot's work cell [16].

Using a base coordinate system has its origin in CNC processes, which most frequently start from a cuboid stock model that is then subtractively processed but is less ideal for free-formed objects without corners that may be referenced.

Finally, hand-eye calibration is computationally and geometrically significantly more complex than either approach [18].

Industry & research teams are already using tools to bridge the gap between the digital and virtual environment in a more efficient way. For example, Antonelli et al uses a motion tracking system to track the 3D positions of an input tool within the robot's workspace [1], the TracePen by Wandelbots allows the user to capture 3D positions as well as its rotation, and the 3D vision module by drag&bot integrates the live 3D point cloud into their programming environment. Such cyber physical systems further act as tools for accessible robot programming by demonstration [14].

Within that context it is important to emphasize that only recent industrial robot systems enable the user to display recorded positions in a 3D environment on the hand controller - rather than just listing their Cartesian positions in textual form - showing the large gap between the state of the art of 3D graphics and industrial applications.

3 RESEARCH GOAL

Augmented reality offers the potential of merging aspects of these approaches and through that making robotic fabrication more accessible for new users. The initial matching of coordinate systems can be simply achieved by manipulating the position and orientation of the robot's 3D model through hand-tracking until it overlaps correctly with the physical robot.

Then, positions may be intuitively defined using the simulated robot within the augmented reality environment, similar to online programming. Finally, if a very high accuracy is needed, the robot's base and tool coordinate systems can be displayed through the AR headset, along with the toolpath to check for general plausibility as well as potential collisions and other issues. The traditional way for

simulation is to bring the manufacturing process into the digital world [9] - using AR gives the possibility to bring the digital code into the physical world.

The fundamental idea of combining augmented reality with industrial robots is not new. A recent review on AR for robot control and planning as well as human-robot-interaction has been provided by Makhataeva and Varol [13]. Until now most research in this field has been made within the context of lightweight collaborative robots, which are also generally based on more recent software architectures, such as collaborative robots [10,17]. However, fabrication still relies primarily on robotic arms that can manipulate large payloads. Research done in AR for large scale industrial robotic arms has been mainly directed towards work cell layout planning, vertical milling and pick-and-place [8,15].

Therefore, this research aims to present a prototypical framework that integrates industrial robot control and simulation into an existing augmented reality framework, so that even non-coders can create customized, augmented reality-supported fabrication processes for robotic arms with high payloads.

Improving the accessibility of robot programming is important in industry due to the increasing proliferation of robots coupled with a lack of programmers. However, most industrial systems focus on the most common application of pick-and-place processes [3].

In fields like the creative industries, there is a great demand for individualized, non-standard processes, often incorporating mass-customization, creating an even higher need to enable users with process and material knowledge, such as carpenters, masons, or even bakers, to apply their expertise through robotics, thus allowing them to offer new, customizable products and to compete with larger companies.

The resulting framework will need to bridge systems with extremely different performance levels and deterministic behavior: The user interacts through and with an augmented reality headset driven by a low-power processor and interfaces with external devices exclusively through its latency-inducing wireless connection. On the other hand, the robot's communication interfaces are meant to connect with deterministic programmable logic controllers (PLCs). This outlines the need for an intermediary system that performs computationally intensive tasks as time-critical communication tasks.

4 PROTOTYPE

The presented prototype has been developed to be used with a combination of a KUKA robot running KSS8.x, a Microsoft HoloLens 2 headset and Unity as the core 3D environment. However, the presented approaches are expected to be applicable to other products as well.

Rather than creating an accessible framework from scratch, we utilize individually proven tools and combine them within the popular game engine Unity: Microsoft's Mixed Reality Toolkit provides an abstraction layer for using the HoloLens 2 headset used in our research, while the author's KUKA|prc (parametric robot control) framework [2] contributes the simulation, programming, and real-time control of KUKA industrial robots.

KUKA|prc originates from the goal of making robotic fabrication accessible to the creative industry but is today used in many other academic and commercial fields. It was initially built upon the visual programming platform Grasshopper, which provides advanced tools for computational geometry and is therefore particularly well suited for defining complex, non-standard processes, such as multi-axis additive fabrication strategies. As a platform-independent .NET library, it has since been brought to other platforms like Autodesk Dynamo and more recently the game engine Unity.

In Unity, a series of nodes have been developed for Unity's Visual Programming system, allowing the user to create motion

sequences, simulate them, and stream them to a robot. However, Unity's geometric library is focused on working with meshes, lacking the analysis functions and accuracy of a NURBS (non-uniform, rational B-splines) modeler that are available within Grasshopper. Therefore, we have developed nodes that can communicate with a Rhino.Compute instance – basically, a local service for processing individual geometric operations or – in this case – entire, serialized Grasshopper definitions.

This enables users with experience in Grasshopper to build upon their existing knowledge of creating geometrically complex parametric definitions that facilitate customization but can augment it with real-time input coming from Unity (Figure 3).

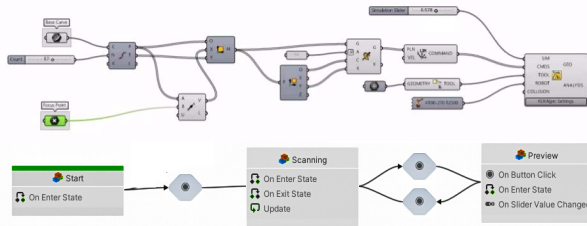


Figure 3: Geometry-focused visual programming using Grasshopper (above), state-focused visual programming using UVS (below).

Within the scope of augmented reality applications, it also offloads computationally intensive tasks from the headset to a local or cloud-based server.

By integrating KUKA|prc through visual programming into Unity, while offloading complex path-planning to an external server, it becomes possible for non-coders to create augmented reality applications for robot control. A major challenge is however still the communication with the robot itself. KUKA provides two optimized interfaces for real-time motion control: the Real-time Sensor Interface (RSI) and mxAutomation. The goal of RSI is to enable a dynamic motion offset at the robot's cycle time of 4ms, for example, towards achieving a constant force over a toolpath based on the data coming from a force sensor, either via an industrial fieldbus or via Ethernet, formatted as XML datagrams – of course, that offset can be expanded to achieve motion control within the robot's entire workspace. On the other hand, mxAutomation is built towards allowing robot control from a PLC and uses the same motion commands as when the robot is processing a control data file. KUKA provides a library so that commands can be sent as UDP datagrams from any PC, rather than binary data from a PLC through an industrial fieldbus. To achieve a smooth motion, mxAutomation needs to buffer several commands in advance of the current one, thereby reducing its responsiveness. In practice, RSI is more suitable for immediate interaction, while mxAutomation can perform accurate fabrication processes.

While mxAutomation is much more lenient when it comes to cycle time, directly controlling the robot through the AR headset proved to be unreliable due to the inherent latency of shared-medium connections like WiFi. This applies even more to RSI, where missed 4ms-cycles have the robot stopping rapidly.

As such, we introduce an intermediate layer based on robust, web-centric SignalR protocol for real-time communication [5]. A .NET application therefore acts as SignalR server for headset communication, Rhino.Compute host for geometry computation, and as a mxAutomation/RSI server that deserializes the JSON data coming from the headset through SignalR and communicates with low latency through Ethernet with the connected industrial robot. Besides setting the relevant IP addresses and ports, no interaction is required from the user.

Therefore, the headset can use web technologies to outsource computation-intensive path-planning by utilizing Rhino.Compute via REST and to control the robot by connecting to a local server via SignalR, which then communicates in low latency with the industrial robot (Figure 4).

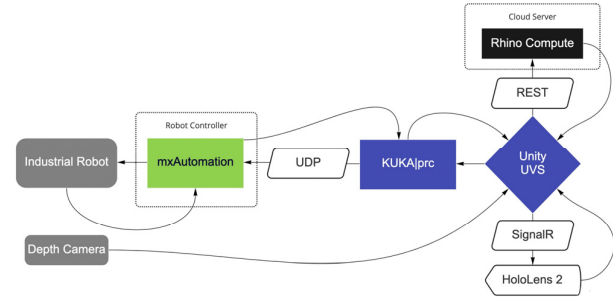


Figure 4: Process architecture from design to fabrication.

These interfaces are accessed via nodes within Unity Visual Scripting (UVS), making it possible for users with little coding expertise to create interactive AR processes with industrial robots.

5 RESULTS

The presented research has been prototypically applied on a mobile additive manufacturing process and a subtractive manufacturing use case, covering common industrial fabrication technologies as a first step towards wider applications of augmented reality.

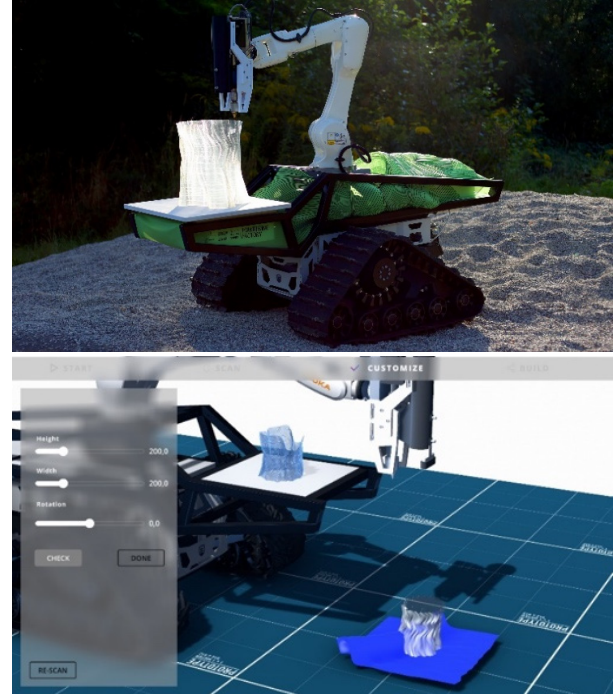


Figure 5: Locally individualized, additive manufacturing using a mobile, robotic platform using Unity and KUKA|prc.

5.1 Use Case: Mobile Additive Manufacturing

The “Wandering Factory” project uses the link between UVS and industrial robots to control an in-situ customized, robotic 3D-printing process on a mobile platform. The robot scans its environment and passes the point cloud to Rhino.Compute, which then calculates the printing toolpath and sends it back to the robot.

The result is a furniture piece that fits precisely onto the scanned surface, using a custom, non-planar 3D printing strategy (Figure 5).

A HoloLens 2 headset can be used to define scanning positions within the robot's workspace, simulate the robot's movement and then sent the serialized movement commands via SignalR to a server that then moves the robot accordingly through the mxAutomation interface, while sending the robot's current position back to the HoloLens 2, to be superimposed over the actual robot with relatively little latency.

5.2 Use Case: Subtractive Manufacturing

For this use case, our UVS component library is extended with a visualization component. This component displays milling toolpaths live over a block of material, allowing us to verify and interact with the robot's trajectory before, during, and after the fabrication process (Figure 6).

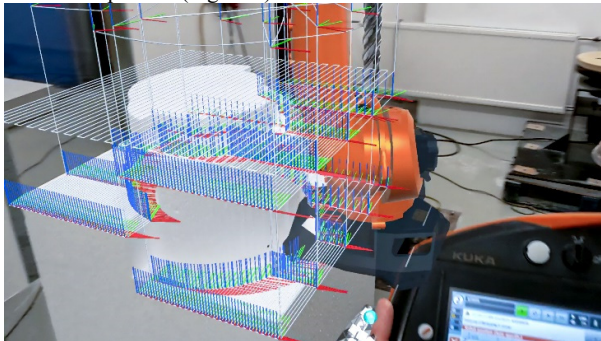


Figure 6: Mixed reality application using a HoloLens 2 headset and a large-scale industrial robot.

We see a great potential in further investigating the integration of augmented reality systems into robotic fabrication processes, with the goal of not only creating accessible ways for the display of and interaction with spatial data but also for defining these interactions in an accessible, task-specific approach that does not require significant programming proficiency.

6 OUTLOOK

The overlying goal of this research is to facilitate the interaction with robotic arms through a low-code approach. Especially the combination of accessible programming with body-tracking and AR has got the potential to allow artisans to apply their tacit body knowledge on tool movement and material behavior more easily to a robotic process. We are aware of the challenges of translating motion from a specific human body to a specific robot kinematic from previous projects - nevertheless, we expect AR to significantly foster the democratization of robotic fabrication.

In our experience, a significant advantage of AR over conventional CAD environments is that it enables users to perceive and optimize "secondary" robotic toolpath parameters such as acceleration and motion blending much more easily. We can imagine extending this approach with an automated generation of a range of motion possibilities, simulating and displaying them on-site in AR and enabling the user to compare them in parallel prior to execution. This could be of specific interest to applications where dynamic acceleration and velocity are essential, such as chiseling or drumming.

REFERENCES

- [1] Dario Antonelli and Sergey Astanin. 2016. Qualification of a Collaborative Human-robot Welding Cell. *Procedia CIRP* 41, (December 2016), 352–357. DOI:https://doi.org/10.1016/j.procir.2015.12.036

- [2] Association for Robots in Architecture. KUKA|prc. Retrieved January 6, 2022 from <https://www.robotsinarchitecture.org/kukaprc>
- [3] J. P. Barreto, F. J.-F. Schöler, and B. Corves. 2017. The Concept of Natural Motion for Pick and Place Operations. In *New Advances in Mechanisms, Mechanical Transmissions and Robotics* (Mechanisms and Machine Science), Springer International Publishing, Cham, 89–98. DOI:https://doi.org/10.1007/978-3-319-45450-4_9
- [4] BEC Rides. *Robotic Entertainment Solutions*. Retrieved from <https://www.bec-rides.com/>
- [5] Johannes Braumann and Karl Singline. 2021. Towards Real-Time Interaction with Industrial Robots in the Creative Industrie. In *2021 IEEE International Conference on Robotics and Automation*, IEEE.
- [6] Sigrid Brell-Cokcan and Johannes Braumann. 2010. A New Parametric Design Tool for Robot Milling. In *LIFE in:formation, On Responsive Information and Variations in Architecture*, 357–363.
- [7] Jonas Buchli, Markus Gifftaler, Nitish Kumar, Manuel Lussi, Timothy Sandy, Kathrin Dörfler, and Norman Hack. 2018. Digital in situ fabrication. *Cement and Concrete Research* 112, (October 2018), 66–75. DOI:https://doi.org/10.1016/j.cemconres.2018.05.013
- [8] J.W.S. Chong, S K Ong, Andrew Nee, and K. Youcef-Youmi. 2009. Robot programming using augmented reality: An interactive method for planning collision-free paths. *Robotics and Computer-Integrated Manufacturing* 25, (June 2009), 689–701. DOI:https://doi.org/10.1016/j.rcim.2008.05.002
- [9] F Hosseinpour and H Hajihosseini. 2009. Importance of simulation in manufacturing. *World Academy of Science, Engineering and Technology* 51, 3 (2009), 292–295.
- [10] Ryan Luke Johns, Jeffrey Anderson, and Axel Kilian. 2020. RoboStim: Modes of Human Robot Collaboration for Design Exploration. In *Impact: Design With All Senses*, Springer International Publishing, Cham, 671–684. DOI:https://doi.org/10.1007/978-3-030-29829-6_52
- [11] Niki Kousi, Christos Gkourmelos, Sotiris Aivaliotis, Christos Giannoulis, George Michalos, and Sotiris Makris. 2019. Digital twin for adaptation of robots' behavior in flexible robotic assembly lines. *Procedia Manufacturing* 28, (January 2019), 121–126. DOI:https://doi.org/10.1016/j.promfg.2018.12.020
- [12] O. N. Krakhmalev, D. I. Petreshin, and O. N. Fedonin. 2017. Mathematical models for base calibration in industrial robots. *Russ. Engin. Res.* 37, 11 (November 2017), 995–1000. DOI:https://doi.org/10.3103/S1068798X17110089
- [13] Zhanat Makhataeva and Huseyin Atakan Varol. 2020. Augmented Reality for Robotics: A Review. *Robotics* 9, 2 (June 2020), 21. DOI:https://doi.org/10.3390/robotics9020021
- [14] Fabian Müller, Christian Deuerlein, and Michael Koch. 2021. Cyber-physical-system for representing a robot end effector. *Procedia CIRP* 100, (January 2021), 307–312. DOI:https://doi.org/10.1016/j.procir.2021.05.071
- [15] Yun Suen Pai, Hwa Jen Yap, Siti Zawiah Md Dawal, S. Ramesh, and Sin Ye Phoon. 2016. Virtual Planning, Control, and Machining for a Modular-Based Automated Factory Operation in an Augmented Reality Environment. *Sci Rep* 6, 1 (June 2016), 27380. DOI:https://doi.org/10.1038/srep27380
- [16] Zengxi Pan, Joseph Polden, Nathan Larkin, Stephen Van Duin, and John Norrish. 2012. Recent progress on programming methods for industrial robots. *Robotics and Computer-Integrated Manufacturing* 28, 2 (April 2012), 87–94. DOI:https://doi.org/10.1016/j.rcim.2011.08.004
- [17] Camilo Perez Quintero, Sarah Li, Matthew KXJ Pan, Wesley P. Chan, H.F. Machiel Van der Loos, and Elizabeth Croft. 2018. Robot Programming Through Augmented Trajectories in Augmented Reality. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1838–1844. DOI:https://doi.org/10.1109/IROS.2018.8593700
- [18] Klaus H. Strobl and Gerd Hirzinger. 2006. Optimal Hand-Eye Calibration. In *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 4647–4653. DOI:https://doi.org/10.1109/IROS.2006.282250
- [19] Andy Zeng, Shuran Song, Johnny Lee, Alberto Rodriguez, and Thomas Funkhouser. 2019. TossingBot: Learning to Throw Arbitrary Objects with Residual Physics. In *Robotics: Science and Systems (RSS)*. Retrieved from <https://tossingbot.cs.princeton.edu/>